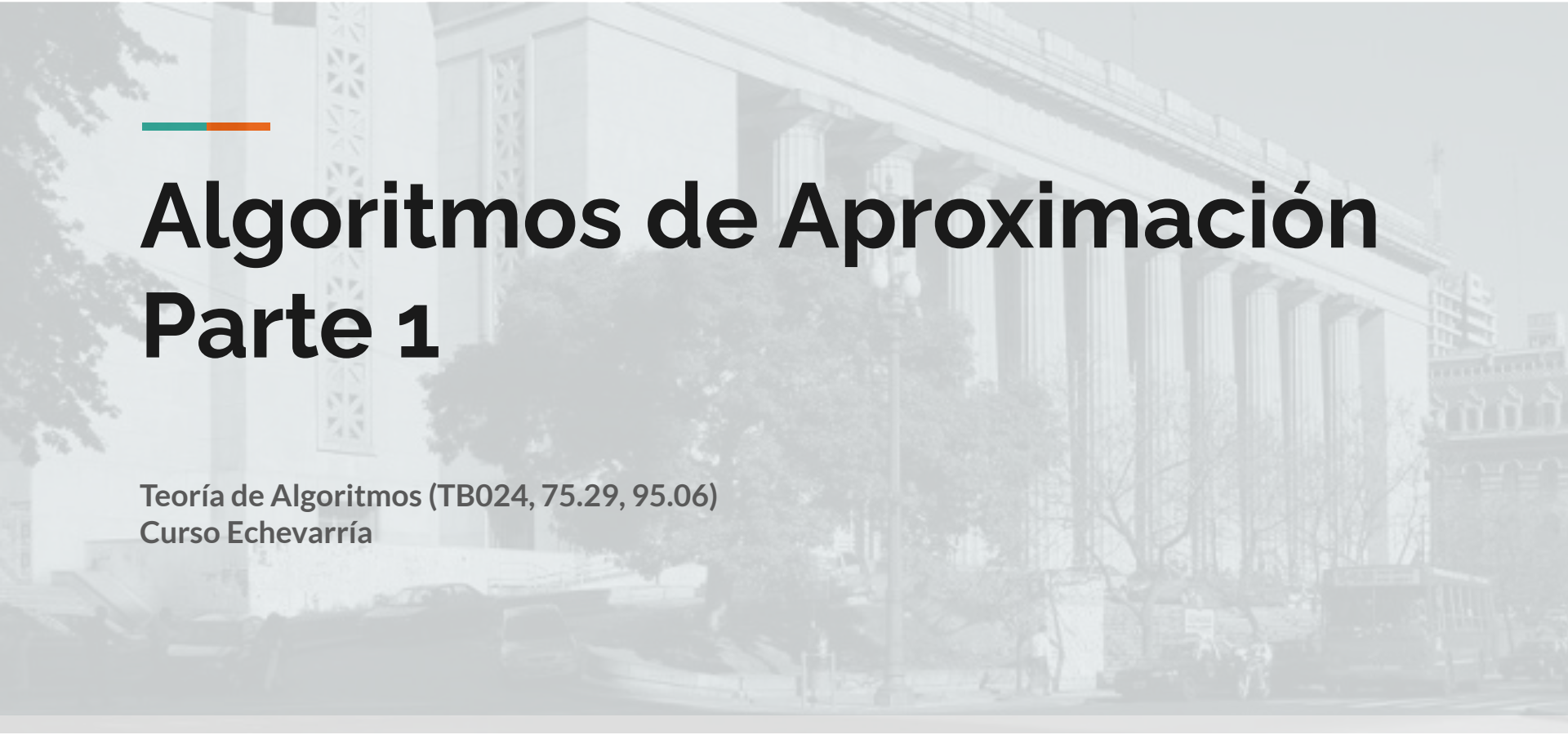
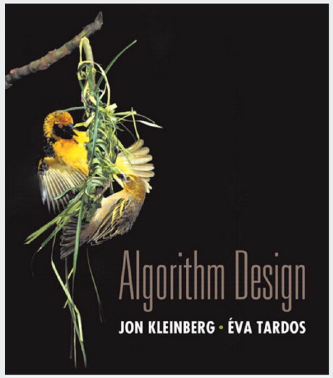


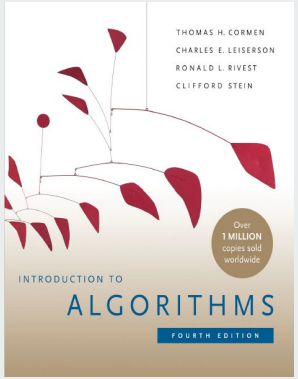
Algoritmos de Aproximación Parte 1

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Introducción



Cap 11



Cap 35

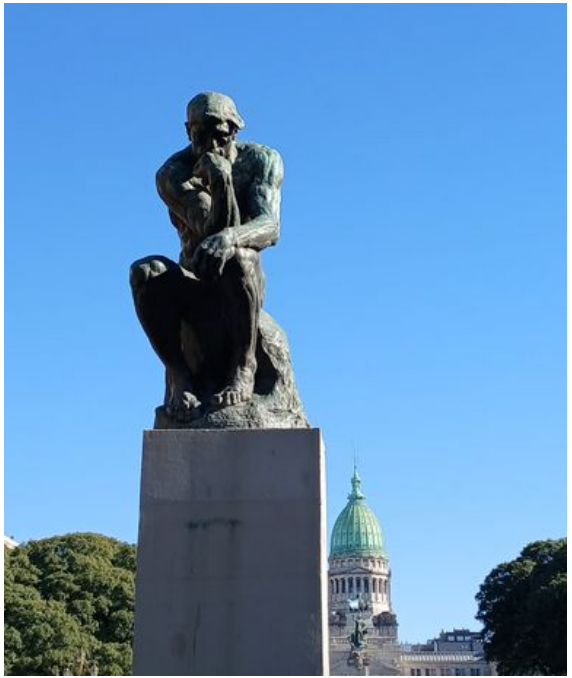


Mikhail Nilov

Contexto y Problemática

Los problemas NP-Completos y NP-Hard son difíciles:

- Encontrar la solución óptima tarda mucho
- ¿Nos podremos conformar con algo menos que la solución óptima?
- ¿Existen garantías sobre lo que encontremos?

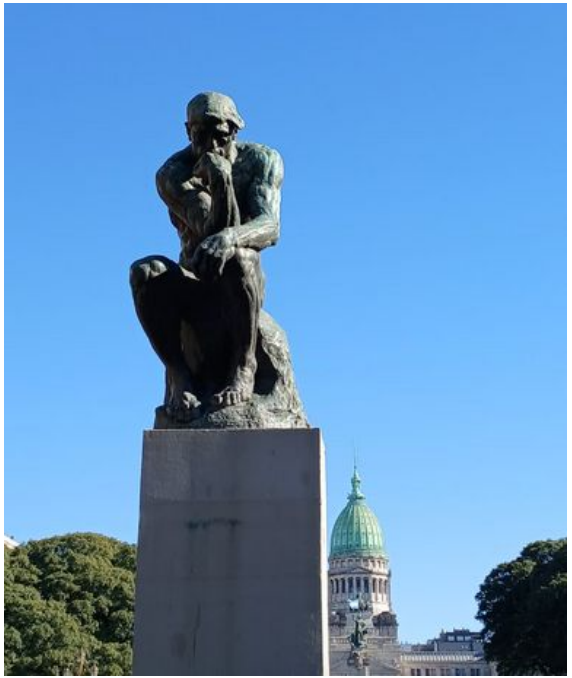


“El Pensador” - A. Rodin

Contexto y Problemática (cont.)

Los algoritmos Greedy podrían servir

- Son fáciles de desarrollar
- Encuentran soluciones rápidamente
- No siempre serán óptimas
- ¿Qué tan buenas puede ser?
- ¿Existen garantías?

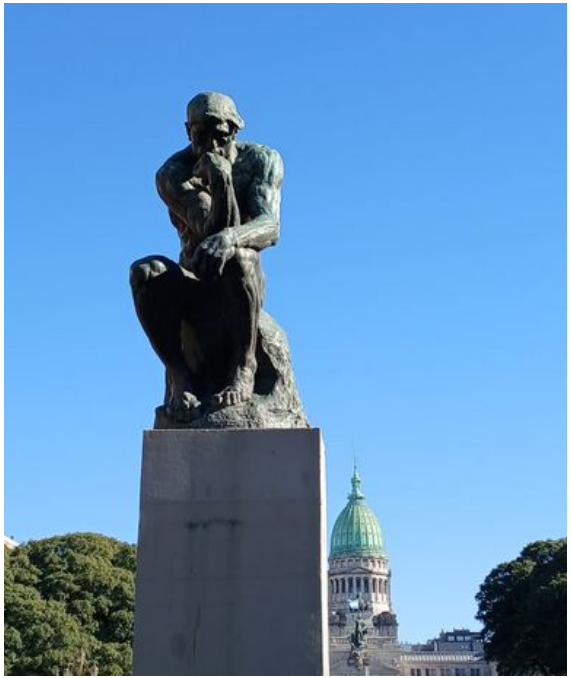


“El Pensador” - A. Rodin

Contexto y Problemática (cont.)

¿Habrá otras herramientas?

- Pricing Method
- Programación Lineal
- Programación Dinámica



“El Pensador” - A. Rodin

Set Cover Aproximado



Pixabay

Set Cover: Definición

- Tenemos un conjunto U de vértices
- Tenemos un conjunto S de subconjuntos de U
- Cada elemento S_i en S tiene un peso w_i
- Un set cover C es una colección de elementos de S tal que su unión es igual a todos los elementos de U

Encontrar un set cover C que minimice el peso total de los elementos de S

Set Cover: Definición (cont.)

Notar que este problema es al menos tan difícil como su problema de decisión:

- Si todos los $w_i=1$, el peso mínimo de un set cover es a lo sumo k sii existe una colección de al menos k conjuntos que cubren U

Otra vez estamos en presencia de un problema NP-Hard:

- Por su complejidad temporal, escala muy mal
- Para un tamaño de U y S suficientemente grandes, resolver el problema de manera óptima es impracticable

Set Cover: Algoritmo

Propongamos un algoritmo Greedy para resolver este problema:
Regla Greedy:

- Asignar el próximo S_i que tenga menor peso?
- Asignar el próximo S_i que cubra más elementos?
- Asignar el próximo S_i que tenga menor costo por elemento ($w_i/|S_i|$)?
- Asignar el próximo S_i que tenga menor costo por elemento cubierto ($w_i/|S_i \cap R|$)



Set Cover: Algoritmo (cont.)

```
def set_cover(U, S, W):
  R=U # elementos de U aún sin seleccionar
  C=set vacío
  Mientras R no esté vacío:
    Agregar a C el S_i con menor w_i/|S_i ∩ R|
    Quitar elementos en S_i de R
  devolver C
```

Set Cover: Algoritmo (cont.)

Ejemplo:

- $U=\{a,b,c,d,e,f,g,h\}$

Solución (parcial) del Algoritmo

- $C=\{\{c,d,g,h\}\}$

Paso 0			
$C=\{\}$; $R=\{a,b,c,d,e,f,g,h\}$			
S_i	w_i	$S_i \cap R$	$w_i/\lvert S_i \cap R\rvert$
{c,d,g,h}	**1**	**{c,d,g,h}**	**0,25**
{a,b,c,d}	1,1	{a,b,c,d}	0,275
{e,f,g,h}	1,1	{e,f,g,h}	0,275
{b,f}	1	{b,f}	0,5
{a}	1	{a}	1
{e}	1	{e}	1

Set Cover: Algoritmo (cont.)

Ejemplo:

- $U=\{a,b,c,d,e,f,g,h\}$

Solución (parcial) del Algoritmo

- $C=\{\{c,d,g,h\},\{b,f\}\}$

Paso 1			
$C=\{\{c,d,g,h\}\}$; $R=\{a,b,e,f\}$			
S_i	w_i	$S_i \cap R$	$w_i/\lvert S_i \cap R\rvert$
~~{c,d,g,h}~~	~~1~~	~~{c,d,g,h}~~	~~0,25~~
{a,b,c,d}	1,1	{a,b}	0,55
{e,f,g,h}	1,1	{e,f}	0,55
{b,f}	**1**	**{b,f}**	**0,5**
{a}	1	{a}	1
{e}	1	{e}	1

Set Cover: Algoritmo (cont.)

Ejemplo:

- $U=\{a,b,c,d,e,f,g,h\}$

Solución (parcial) del Algoritmo

- $C=\{\{c,d,g,h\},\{b,f\},\{a\}\}$

Paso 2			
$C=\{\{c,d,g,h\},\{b,f\}\}$; $R=\{a,e\}$			
S_i	w_i	$S_i \cap R$	$w_i/\lvert S_i \cap R\rvert$
~~{c,d,g,h}~~	~~1~~	~~{c,d,g,h}~~	~~0,25~~
{a,b,c,d}	1,1	{a}	1,1
{e,f,g,h}	1,1	{e}	1,1
~~{b,f}~~	~~1~~	~~{b,f}~~	~~0,5~~
{a}	**1**	**{a}**	**1**
{e}	1	{e}	1

Set Cover: Algoritmo (cont.)

Ejemplo:

- $U=\{a,b,c,d,e,f,g,h\}$

Solución (parcial) del Algoritmo

- $C=\{\{c,d,g,h\},\{b,f\},\{a\},\{e\}\}$

Paso 3			
$C=\{\{c,d,g,h\},\{b,f\},\{a\}\}$; $R=\{e\}$			
S_i	w_i	$S_i \cap R$	$w_i/\lvert S_i \cap R\rvert$
~~{c,d,g,h}~~	~~1~~	~~{c,d,g,h}~~	~~0,25~~
{a,b,c,d}	1,1	{a}	1,1
{e,f,g,h}	1,1	{e}	1,1
~~{b,f}~~	~~1~~	~~{b,f}~~	~~0,5~~
~~{a}~~	~~1~~	~~{a}~~	~~1~~
{e}	1	{e}	1

Set Cover: Algoritmo (cont.)

Ejemplo:

- $U=\{a,b,c,d,e,f,g,h\}$

Solución del Algoritmo

- $C=\{\{c,d,g,h\},\{b,f\},\{a\},\{e\}\}$
- $W=4$

Solución Óptima:

- $C^*=\{\{abcd\}, \{e,f,g,h\}\}$
- $W^*=2,2$

Paso 4			
$C=\{\{c,d,g,h\},\{b,f\},\{a\}\}$; $R=\{\}$			
S_i	w_i	$S_i \cap R$	$w_i/\lvert S_i \cap R\rvert$
~~{c,d,g,h}~~	~~1~~	~~{c,d,g,h}~~	~~0,25~~
{a,b,c,d}	1,1	{a}	1,1
{e,f,g,h}	1,1	{e}	1,1
~~{b,f}~~	~~1~~	~~{b,f}~~	~~0,5~~
~~{a}~~	~~1~~	~~{a}~~	~~1~~
~~{e}~~	~~1~~	~~{e}~~	~~1~~

Set Cover: Conclusiones

- Nuestro algoritmo efectivamente construye un set cover
- ¿Cuánto más grande puede ser el peso W del set cover que encuentra nuestro algoritmo respecto del óptimo W^*?
- Se demuestra que $W \leq H(d^*)\ W^*$, con $d^* = max_i\ |S_i|$ y $H(d^*) = \sum_{i=1}^{d^*} \frac{1}{i}$

- Como $H(d^*)$ está acotada por $log(d^*)$, W será a lo sumo un factor $log(d^*)$ de W^*
- Corolario: cuanto más pequeños sean los S_i, mejor garantía tendrá nuestro algoritmo

Vertex Cover Aproximado



Rudy Kirchner

Vertex Cover: Definición

- En un grafo $G = (V,E)$, un subconjunto S de vértices cubre al grafo si todas las aristas del grafo tienen al menos un extremo en S
- El problema consiste en encontrar el subconjunto de vértices más chico posible que cubra todas las aristas

Dado un grafo G y un número k,
¿Existe en G un Vertex Cover de un tamaño de a lo sumo k?

Vertex Cover: Definición (cont.)

Vamos a incorpora a Vertex Cover una pequeña variante:

- En un grafo $G = (V,E)$, un subconjunto S de vértices cubre al grafo si todas las aristas del grafo tienen al menos un extremo en S
- **Cada vértice $v_i \in V$ tiene un peso $w_i \geq 0$**
- El problema consiste en encontrar el subconjunto de vértices ***de menor peso*** *que cubra todas las aristas*

Dado un grafo G,y los pesos de los vértices,
*¿Cuál es el Vertex Cover **de menor peso**?*

Vertex Cover $\leq_P$ Set Cover

Antes de seguir, una reducción rapidita: Vertex Cover $\leq_P$ Set Cover

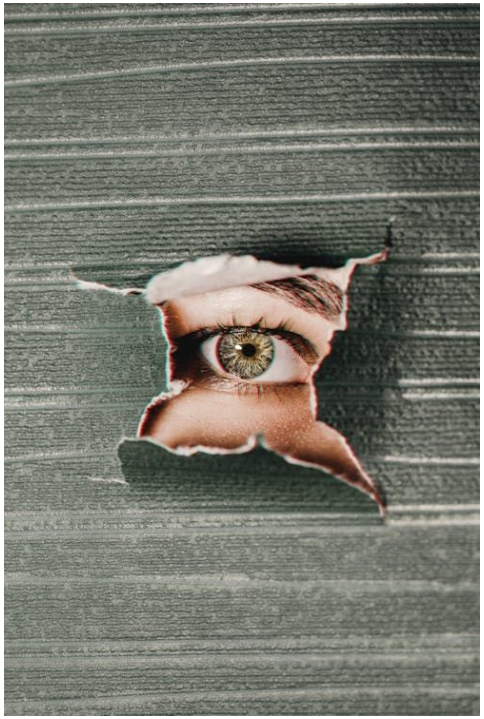
VC	Transformación Entrada	SC	Transformación Salida	Solución VC
Grafo G=(V,E) inicial, y tamaño k para el que se quiere saber si existe VC	• U = E • Para cada vértice v_i, definir un S_i cuyos elementos son los e_i incidentes a v_i • w_i=1 para todo e_i	Resolver Set Cover con U, S y k	Existe/No Existe Set Cover de tamaño k	Existe/No Existe VC de tamaño k

Vertex Cover: Aproximación

- Ya sabemos que Vertex Cover se puede reducir a Set Cover
- Ya conocemos un algoritmo de aproximación para resolver Set Cover
- La pregunta que cabe formularse es…

¿Podremos reutilizarlo total o parcialmente para resolver Vertex Cover?

Vertex Cover: Aproximación (cont.)



Jonathan Borba

¡Ojo!

- La reducción que hicimos fue para el problema de decisión

¿Realmente se puede usar el algoritmo de Set Cover para resolver Vertex Cover?

Vertex Cover: Aproximación (cont.)

Sí, claro que se puede:

- Supongamos que tenemos una instancia del Vertex Cover ponderado, $G=(V,E)$
- Construyamos una instancia de Set Cover de la siguiente manera:
  - U es igual a E
  - Para cada $v_i \in V$, definimos un S_i con todas las aristas que inciden en v_i, y le damos a S_i un peso igual a w_i
- De esta forma:
  - Una colección de S que cubre a U es un Vertex Cover de G
  - El máximo tamaño de todos los S_i es el grado d de Set Cover
- Finalmente, podemos usar el algoritmo de Set Cover para resolver Vertex Cover

Vertex Cover: Aproximación (cont.)



Linda Eller-Schein

- - = = ¡¡Mucho Cuidado!! = = - -

- No siempre se puede usar una reducción para diseñar un algoritmo de aproximación
- En el caso de VC y SC funcionó, pero nos lo aseguramos a través de una buena demostración

Ejemplo:

- Probamos que Independet Set $\leq_P$ Vertex Cover
- En este caso, no podemos usar un algoritmo de aproximación de VC para resolver IS:
  - Si bien un conjunto de vértices I es independiente sii su complemento S=V-I es VC, ¿qué sería el complemento de un VC aproximado?

Programación Lineal



Cottonbro Studio

Programación Lineal: Vertex Cover

Construyamos un modelo de Programación Lineal para Vertex Cover

- Variables:
  - X_i: vale 1 si se selecciona el vértice i, y 0 si no; binaria
- Constantes:
  - El peso w_i de cada vértice
- Restricciones:
  - Para cada arista (i,j) , al menos una de X_i o X_j debe quedar seleccionada
- Función Objetivo:
  - Minimizar la sumatoria de $w_i X_i$

Programación Lineal: Vertex Cover (cont.)

Modelo de Programación Lineal:

- Para cada arista (i,j):
  - $X_i + X_j \geq 1$
- Función Objetivo:
  - $Z(min)=\sum w_i X_i$

Interpretación del resultado:

- S será el conjunto de vértices v_i para los que X_i valga 1
- $w(S)$ será el valor que alcance la función objetivo

Programación Lineal: Vertex Cover (cont.)



Linda Eller-Schein

Demasiado fácil: ¿dónde está la trampa?

- Nos estamos olvidando de que las variables X_i no pueden ser continuas: deben ser enteras
- Si fueran continuas, una solución donde $X_1 = 0,3$ y $X_2 = 0,7$ podría ser factible si $X_1 + X_2 \geq 1$

Este es un problema de **Programación Lineal Entera**

- Y eso es más difícil de resolver que un problema de Programación Lineal Continua (que se demostró que es Polinomial)
- De hecho, PLE está en la lista de 21 problemas NP-Completos de Karp

Programación Lineal: La Relajación Lineal

Veamos qué pasa si intentamos resolver el modelo de PLE usando variables continuas:

- A esto se le llama **Relajación Lineal**
- Consiste en quitarle a las variables enteras su restricción de integralidad
- Debemos estar preparados para cosas extrañas:
  - Ya no tenemos sólo 3 posibilidades para satisfacer $X_1+X_2 \geq 1$: ahora hay infinitas posibilidades
  - Si la solución de la RL es $X_1=1$ o $X_1=0$, es fácil, pero…
  - ¿Qué quiere decir que $X_1=0{,}5$? ¿Es lo mismo que $X_1=1$? ¿O tendría que ser más bien $X_1=0$?

Programación Lineal: La Relajación Lineal

Hay algo seguro: el óptimo de la RL será “mejor” que el óptimo entero

- Si $w(S)$ es el óptimo de la Relajación Lineal y $w(S^*)$ es el óptimo de Vertex Cover, entonces $w(S) \leq w(S^*)$
- Esto es porque la RL es menos restrictiva que el problema PLE original
- Ergo, el valor del funcional de la RL tiene que ser igual o mejor que el de PLE

Programación Lineal: La Relajación Lineal

¿Cómo hacemos para encontrar una solución aproximada usando la RL?

- El abordaje natural es redondear los valores de X_i
- Dada una solución fraccionaria $\{X_i\}$, definimos que $S=\{i \in V: X_i \geq ½\}$
- ¿Es S un Vertex Cover?
  - Para toda arista (i,j), tanto X_i como X_j deben cumplir que $X_i+X_j \geq 1$
  - Así que alguna de las dos tiene que ser mayor o igual que $½$
  - Entonces, alguna de las dos variables será redondeada hacia arriba
  - Por lo tanto, i o j (o ambas) pertenecerán a S: es un vertex cover

Programación Lineal: La Relajación Lineal

¿Cómo hacemos para encontrar una solución aproximada usando la RL?

- ¿Se puede acotar $w(S)$?
  - Cada i en S tendrá asociada una $X_i \geq ½$
  - El aporte de cada vértice i al peso total estará entre $½\ w_i$ y w_i (y no más, porque ya no le convendrá al funcional)
  - Nos queda: $$\underbrace{\sum_i w_i X_i}_{\text{Z RL}} \geq \underbrace{\sum_{i \in S} w_i X_i}_{\text{Z para vértices en S}} \geq \frac{1}{2} \underbrace{\sum_{i \in S} w_i}_{\text{Peso de S}} = \frac{1}{2} \underbrace{w(S)}_{\text{Peso de S}}$$

- El peso de nuestra solución será a lo sumo el doble del peso del óptimo de la RL
- Y también del óptimo entero de PLE (reemplace la X_i continua por la X_i entera, y las desigualdades se siguen cumpliendo)

Programación Lineal: Conclusiones

- Podremos modelar muchos problemas con PLE
- Resolverlos, es otra historia:
  - Su dificultad será NP-Completa o NP-Hard, según el tipo de problema
- Con la Relajación Lineal podemos obtener una solución aproximada en tiempo polinomial
  - Modificando solamente el tipo de variables
- En los casos donde relajemos variables binarias será más fácil encontrar las cotas de la aproximación

Problema de la Mochila





El problema de la mochila - Greedy

- Para cada elemento, determinamos el índice Pi/Wi
- Ordenamos la lista según ese índice.
- Recorremos la lista en orden agregando los elementos que se puedan.

- Es muy rápido: O (n.log n)
- Suele ser bueno, pero puede ser arbitariamente malo.
- ¿Podemos acotar el desvío respecto del óptimo?

El problema de la mochila - Garantía de calidad

- Todo gira en torno al **elemento crítico**: Es el primero que no entra en la mochila.
- Según el índice, todos los que quedaron adentro son “mejores”, y todos los que quedaron afuera son “peores”.



El problema de la mochila - Garantía de calidad

- Sea:
  - Z: Solución óptima
  - Z': Solución de la heurística greedy
  - Pc: Beneficio del elemento crítico
- Sabemos que:
  - $Z < Z' + Pc$
- Nuestro nuevo algoritmo Greedy 2.0 ® :
  - $Z'' = Max(Z', Pc)$
- **Tiene garantía de calidad 0,5**
- Si $Z' + Pc$ es mayor que $Z \rightarrow$ Al menos uno de los dos es mayor que $Z/2$



El problema de la mochila - Cotas

- La garantía de calidad nos da un límite teórico, y suele ser amplio.
- En general, queremos saber qué tan lejos del óptimo está UNA SOLUCIÓN EN PARTICULAR.
- Para poder decir cosas como “Si está a más del 5%, sigo buscando otra solución mejor.”



El problema de la mochila - Cotas

- Una cota inferior es cualquier solución válida.
- La primera cota superior es la relajación lineal: Resolver el problema con PL como si las variables fueran continuas.





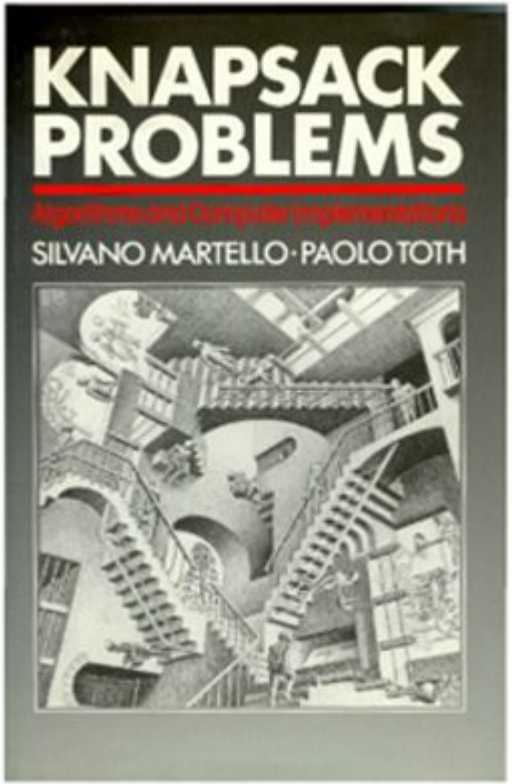
El problema de la mochila - Cotas

- Martello y Toth (1990) mejoraron esta cota.
- En vez de partir el elemento crítico, plantearon dos alternativas: El elemento crítico se incluye (vale 1) o no se incluye (vale 0)

$$W_1 = \sum_{i=1}^{s-1} p_i + \left\lfloor \bar{c} \frac{p_{s+1}}{w_{s+1}} \right\rfloor$$

$$W_2 = \sum_{i=1}^{s-1} p_i + \left\lfloor p_s - (w_s - \bar{c}) \frac{p_{s-1}}{w_{s-1}} \right\rfloor$$

$$U_2 = max(W_1; W_2)$$



Prob. de la mochila - Esquema de aproximación

- Sahni (1975) propuso un primer esquema de aproximación.

```
procedure GS:
input: n . c . (p_j) . (w_j) . M ;
output: z^g . X ;
begin
    z^g := 0;
    ĉ := c − Σ_{j∈M} w_j ;
    X := Ø;
    for j := 1 to n do
        if j ∉ M and w_j ≤ ĉ then
            begin
                z^g := z^g + p_j ;
                ĉ := ĉ − w_j ;
                X := X ∪ {j}
            end
end.
```

- Parte de la lista ya ordenada.
- Agrega sí o sí los primeros M elementos.
- Prueba los restantes, igual que greedy

Prob. de la mochila - Esquema de aproximación

```
procedure S(k):
input: n, c, (p_j), (w_j);
output: z^h, X^h;
begin
    z^h := 0;
    for each M ⊂ {1, ..., n} such that |M| ≤ k and Σ_{j∈M} w_j ≤ c do
        begin
            call GS;
            if z^g + Σ_{j∈M} p_j > z^h then
                begin
                    z^h := z^g + Σ_{j∈M} p_j;
                    X^h := X ∪ M
                end
        end
end.
```

Martello y Toth (1990)

- Probamos diferentes M: todos los conjuntos posibles de hasta k elementos.
- ¿Cuál es la complejidad?
- O (n ^ (k+1))
- La solución está, en el peor caso a r=k/(k+1) del óptimo.
- Es un esquema de aproximación polinomial: Para cualquier distancia r el algoritmo está acotado polinomialmente.

Problema de la mochila: Conclusiones

- El algoritmo greedy básico puede dar muy mal.
- Se puede aplicar una garantía de calidad 0,5
- Cota 1: Relajación lineal
- Cota 2: Martello y Toth: Elemento crítico sí o no
- Esquema de aproximación polinomial: Si fijo el margen de error, es polinomial. (Sahni 1975)

Problema del Balanceo de Carga



Hudson Apunyo

Balanceo de Carga: Definición

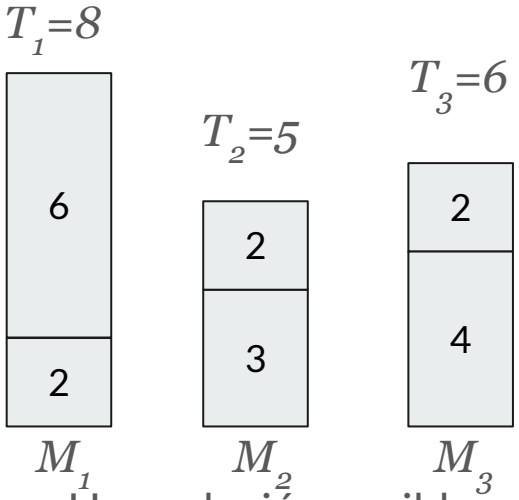
Se tiene un conjunto de m máquinas $M_1, M_2, \ldots, M_m$ y un conjunto de n tareas. Cada tarea j tiene un tiempo de procesamiento t_j.

Se desea asignar las tareas a las máquinas de forma tal que las cargas ubicadas en todas las máquinas estén lo más "balanceadas" que sea posible

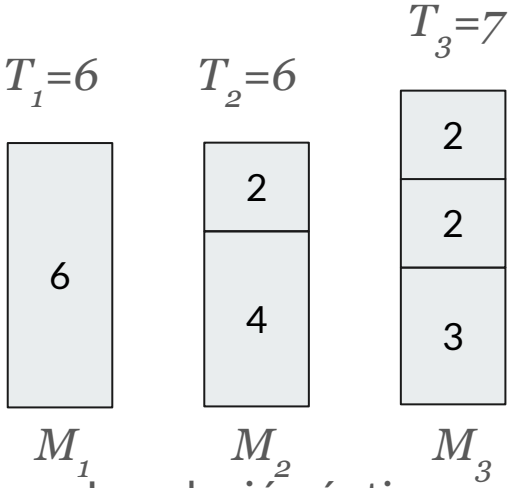
Balanceo de Carga: Definición (cont.)

Ejemplo:

- Máquinas: M_1, M_2, M_3
- Tareas: $\{2,3,4,6,2,2\}$



Una solución posible



La solución óptima

Balanceo de Carga: Definición (cont.)

- Definamos a $A(i)$ como el conjunto de tareas asignadas a la máquina M_i
- El tiempo que trabaja la máquina M_i (su carga) es:

$$T_i = \sum_{j \in A(i)} t_j$$

El problema consiste en minimizar el máximo T_i de todas las máquinas (y es NP-Hard):

- Para una cantidad de tareas y máquinas suficientemente grande, resolver el problema de manera óptima es impracticable

Balanceo de Carga: Algoritmo

Propongamos un algoritmo Greedy para resolver este problema:
Regla Greedy:

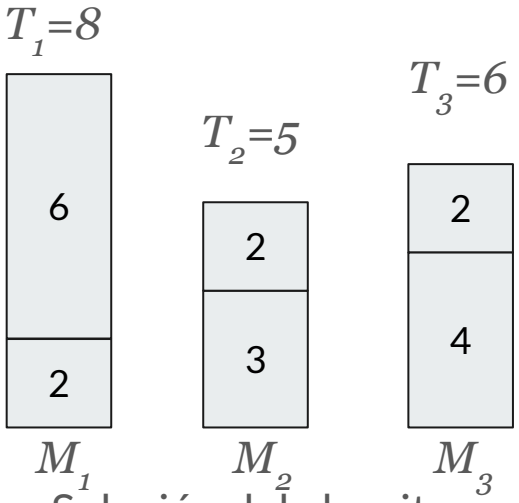
- Asignar el próximo trabajo a la máquina con menor carga

```
def balanceo(tiempos):
  T(i)=0 para todas las máquinas i
  A(i)={} para todas las máquinas i
  n=len(tiempos)
  para j=1..n:
    i=índice de la máquina con menor T(i)
    A(i)=A(i) U {j}
    T(i)= T(i)+tiempos(j)
```

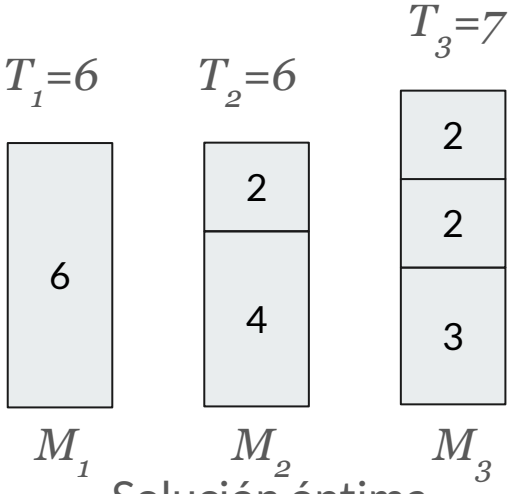
Balanceo de Carga: Algoritmo (cont.)

Ejemplo:

- Máquinas: M_1, M_2, M_3
- Tareas: $\{2,3,4,6,2,2\}$



Solución del algoritmo



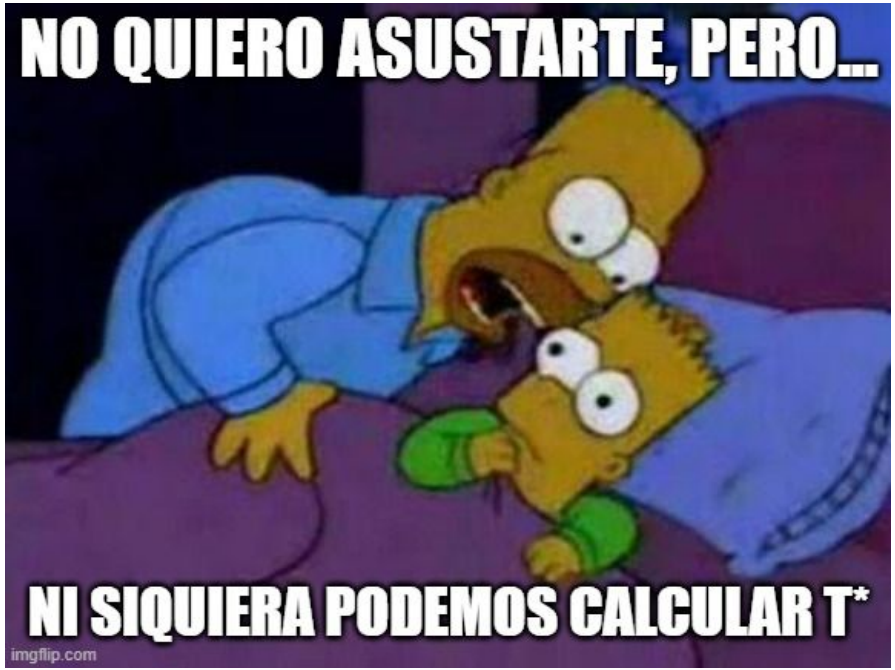
Solución óptima

Balanceo de Carga: Análisis

- Como era de esperar, nuestro algoritmo no siempre da la solución óptima
- Definamos T como el tiempo que trabajó la máquina con mayor carga en la asignación obtenida con el algoritmo Greedy: $T=max\{T_i\}$
- Definamos T^* como el tiempo que trabajó la máquina con mayor carga pero en la solución óptima

Balanceo de Carga: Análisis (cont.)

Veamos si podemos acotar a T respecto de T*



Balanceo de Carga: Análisis (cont.)

Empecemos con una cota mínima:

- Cualquiera de las máquinas tiene que hacer por lo menos una fracción 1/m del total del trabajo a asignar
- El máximo T* es por lo menos una fracción 1/m del trabajo total

$$T^* \geq \frac{1}{m} \sum_j t_j$$

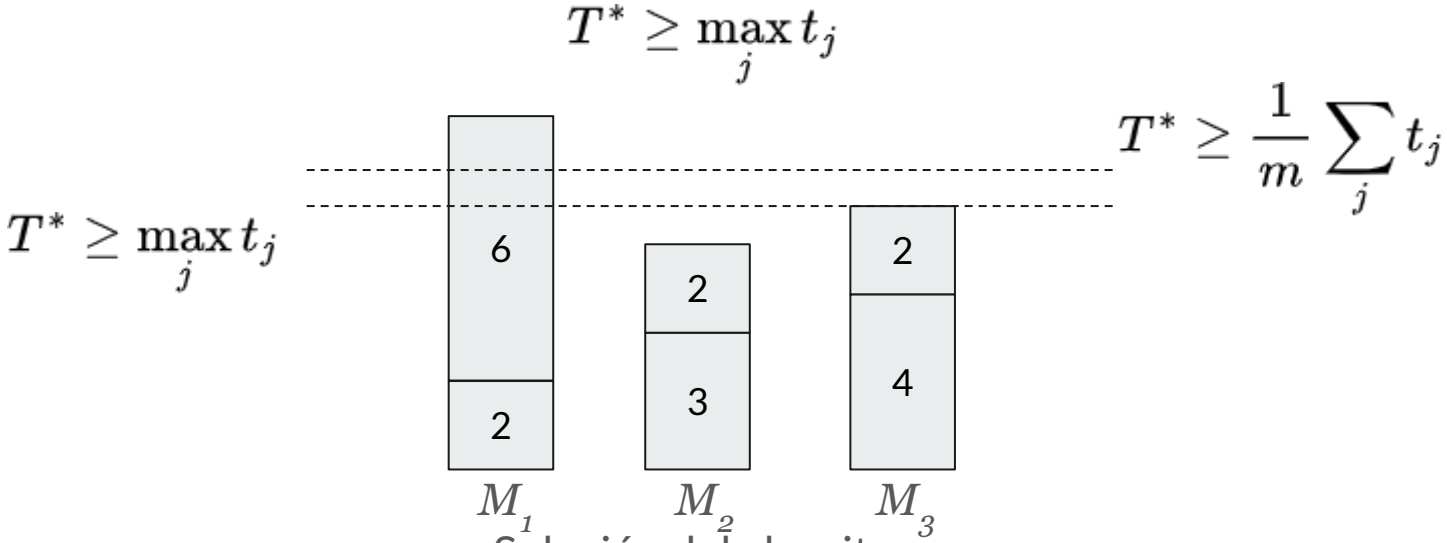
Pero esta cota es débil:

- Supongamos que hay un trabajo mucho más largo que el resto
- El algoritmo podría asignarlo a una máquina dedicada exclusivamente
- Sería el último trabajo en terminar
- El algoritmo produciría una asignación óptima
- Pero la cota anterior estaría muy lejos del óptimo

Balanceo de Carga: Análisis (cont.)

Ok, definamos esta otra cota mínima:

- El máximo T^* es por lo menos mayor al máximo t_i

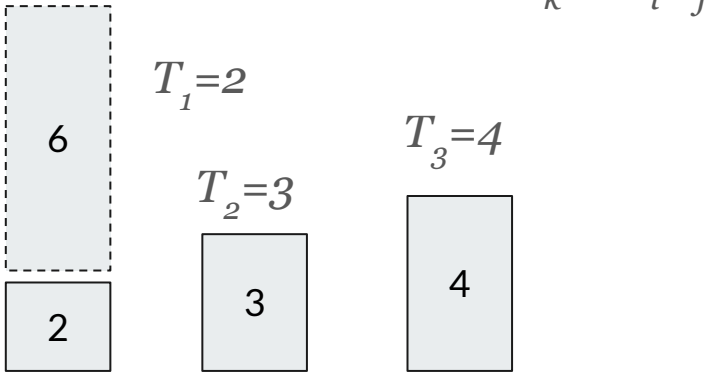


Solución del algoritmo

Balanceo de Carga: Análisis (cont.)

Miremos la máquina M_i que alcanzó la mayor carga, cuando se asignó su última tarea:

- Cuando se asignó j a M_i, ésta era la que tenía la menor carga hasta ese momento
- Esa carga era T_i-t_j
- El resto de las máquinas tenía una carga $T_k \geq T_i$-t_j



Balanceo de Carga: Análisis (cont.)

- Carga T de todas las máquinas hasta ese momento: $\sum_k T_k \geq m(T_i - t_j)$
- O bien: $T_i - t_j \leq \frac{1}{m} \sum_k T_k$
- La sumatoria en en el miembro derecho es la primer cota mínima que encontramos: $T_i - t_j \leq \frac{1}{m} \sum_k T_k \leq T^*$

$$\therefore T_i - t_j \leq T^* (1)$$

- Veamos el tiempo t_j y la segunda cota: $t_j \leq \max_j t_j \leq T^*$

$$\therefore t_j \leq T^* (2)$$

- Sumando (1) y (2): $T_i = (T_i - t_j) + t_j \leq 2T^*$

Balanceo de Carga: Análisis (cont.)

Como el tiempo máximo es $T = T_i$, encontramos una cota máxima para la solución del algoritmo: $T \leq 2T^*$

Balanceo de Carga: Mejora

¿Podremos obtener una mejor cota máxima?

¿Se puede mejorar el algoritmo que tenemos?

Podríamos ordenar las tareas por su duración:

- Si hay una tarea mucho más grande que el resto, se asigna a una máquina en exclusiva

Balanceo de Carga: Mejora (cont.)

```
def balanceo_ordenado(tiempos):
  T(i)=0 para todas las máquinas i
  A(i)={} para todas las máquinas i
  Ordenar tiempos en orden decreciente
  n=len(tiempos)
  para j=1..n:
    i=índice de la máquina con menor T(i)
    A(i)=A(i) U {j}
    T(i)= T(i)+tiempos(j)
```

Balanceo de Carga: Mejora (cont.)

Notar lo siguiente:

- Si hubiera m tareas o menos, el algoritmo siempre devolvería una asignación óptima
  - Entonces, $T=T^*$
- Si hubiera más de m tareas:
  - Las primeras m tareas tendrían una duración de al menos t_{m+1}
  - Al asignar la tarea $m+1$, habrá una máquina que quedará con dos tareas asignadas
  - Así que esta máquina tendrá una carga $T_i \geq 2\, t_{m+1}$
  - Entonces:

$$T^* \geq 2t_{m+1}$$

$$\therefore \frac{1}{2}T^* \geq t_{m+1} \quad (3)$$

Balanceo de Carga: Mejora (cont.)

Miremos la máquina M_i que alcanzó la mayor carga:

- Si tiene una sola tarea asignada, la asignación es óptima: $T=T^*$
- Si tiene al menos dos tareas asignadas:
  - Supongamos que t_j fue la última tarea que se le asignó
  - Notar que $j \geq m+1$ porque ya se asignaron las primeras m tareas a m máquinas diferentes
  - Entonces $t_j \leq t_{m+1}$, y por (3): $t_j \leq t_{m+1} \leq \frac{1}{2}T^* \quad (4)$

- Sumando (1) de la demostración del algoritmo original y (4):

$$T_i \leq \frac{3}{2}T^*$$

$$\therefore T \leq \frac{3}{2}T^*$$

Balanceo de Carga: Mejora (Cont.)

Resumiendo:

- Algoritmo original: $T \leq 2\ T^*$
- Algoritmo mejorado: $T \leq 3/2\ T^*$

Balanceo de Carga: Conclusiones

- Dado un problema NP-Hard, sabemos que encontrar la solución óptima puede ser difícil o computacionalmente muy costoso
- Un algoritmo Greedy nos puede dar una solución rápidamente aunque, (muy) probablemente, subóptima
- Con un poco de maña, podemos encontrar una “garantía de calidad” de la solución
- Y con un poco más de ingenio, podemos mejorar esa “garantía”



¡Muchas gracias!